Git Notes

Vincenzo Brachetta

June 2026

Introduction

Git is a distributed version control system used to track changes in files and collaborate on software projects, documents, and more.

Key uses:

- Tracking changes over time
- Collaborating with others
- Maintaining backups
- Supporting reproducible workflows

Basic Concepts

Term	Meaning
Repository (repo)	A project folder managed by Git
Commit	A saved snapshot of changes
Branch	An independent line of development
Clone	A copy of a repository
Push	Uploading changes to a remote server
Pull	Downloading changes from a remote server
Remote	An online repository (e.g. GitHub, GitLab)
Stash	Temporary storage for uncommitted changes
Tag	A named reference to a specific commit

Installing Git

On Debian:

```
sudo apt update && sudo apt install git
```

Verify the installation:

```
git --version
```

Initial Configuration

Set your name and email once after installation. These are embedded in every commit.

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Set a preferred editor (e.g. VS Code):

```
git config --global core.editor "code --wait"
```

Check your configuration:

```
git config --list
```

SSH Authentication

Using SSH avoids entering your password on every push or pull.

Generate a key pair:

```
ssh-keygen -t ed25519 -C "your.email@example.com"
```

Copy the public key and add it to your GitHub or GitLab account:

```
cat ~/.ssh/id_ed25519.pub
```

Test the connection:

```
ssh -T git@github.com
```

> **Note:** All remote URLs in this document use the SSH format (`git@host:user/repo.git`). Ensure your SSH key is configured before cloning or adding remotes.

Creating a Repository

Navigate to your project folder and initialise Git:

```
cd my-project
git init
```

This creates a hidden `.git` directory that stores all version history.

Checking Status

```
git status
```

Shows modified, untracked, and staged files, as well as the current branch. Run this often.

Staging Files

Stage a single file:

```
git add script.py
```

Stage all changes:

```
git add .
```

Stage changes interactively (review each change before staging):

```
git add -p
```

Creating Commits

```
git commit -m "Brief description of what changed"
```

Good Commit Messages

- `Add data loading function`
- `Fix off-by-one error in loop`
- `Update configuration file`
- `Remove unused imports`

Avoid vague messages such as `fix`, `update`, or `final version`.

Viewing History

```
git log --oneline
git log --oneline --graph --all
git log
```

Show changes introduced by a specific commit:

```
git show <commit-hash>
```

Ignoring Files

Create a `.gitignore` file in the root of your repository:

```
__pycache__/
*.pyc
*.tmp
*.log
.env
.vscode/
```

Branches

Create and switch to a new branch:

```
git switch -c new-feature
```

Switch to an existing branch:

```
git switch main
```

List branches:

```
git branch          # local branches
git branch -a       # includes remote branches
```

Delete a branch:

```
git branch -d new-feature
```

Merging

```
git switch main
git merge new-feature
```

Resolving Conflicts

If a conflict occurs, open the affected file and resolve manually:

```
<<<<<<< HEAD
x = compute(a)
=======
x = compute(b)
>>>>>>> new-feature
```

Then stage and commit:

```
git add script.py
git commit -m "Resolve merge conflict"
```

Remote Repositories

Clone a repository:

```
git clone git@github.com:user/project.git
```

Add a remote:

```
git remote add origin git@github.com:user/project.git
```

Check remotes:

```
git remote -v
```

Push and Pull

Push (first time, sets upstream):

```
git push -u origin main
```

Push (subsequent):

```
git push
```

Pull:

```
git pull
```

Fetch (download without merging):

```
git fetch origin
```

Undoing Changes

Action	Command
Unstage a file	`git restore --staged file.py`
Discard local changes	`git restore file.py`
Amend last commit	`git commit --amend`
Undo a commit (safely)	`git revert <commit-hash>`

> **Warning:** Avoid `git reset --hard` on shared branches as it rewrites history.

Viewing Differences

```
git diff                  # unstaged changes
git diff --staged         # staged changes
git diff commit1 commit2  # between two commits
```

Stashing

Temporarily save unfinished work:

```
git stash
git stash pop
```

Tags

Mark a specific point in history:

```
git tag v1.0
git tag                    # list tags
git push origin --tags
```

Daily Workflow

```
git pull
# make changes
git status
git add .
git commit -m "Describe what changed"
git push
```

Command Summary

Command	Purpose
`git init`	Initialise a repository
`git status`	Check current state
`git add .`	Stage all changes
`git commit -m "msg"`	Save a snapshot
`git log --oneline`	View compact history
`git branch`	List branches
`git switch -c name`	Create and switch branch
`git merge name`	Merge a branch
`git push`	Upload changes
`git pull`	Download changes
`git clone URL`	Clone a repository
`git diff`	View changes
`git stash`	Save uncommitted changes
`git tag v1.0`	Tag a commit

Good Practices

- Commit frequently with clear messages
- Use branches for new features or experiments
- Never commit passwords, API keys, or large data files
- Keep a `.gitignore` in every repository
- Always pull before starting new work

Resources

- Official Git Documentation

- Pro Git Book (free)
- GitHub Docs
- GitLab Docs